

Captura, Herencia y Gestión de Tokens de API por Agentes y Subagentes en el Entorno de Microsoft Windows

La arquitectura de ejecución de agentes autónomos y subagentes en el sistema operativo Microsoft Windows presenta particularidades fundamentales en la forma en que los entornos de ejecución (runtimes), las librerías de orquestación y las herramientas de línea de comandos gestionan, heredan y aíslan las credenciales de autenticación. A diferencia de los entornos de tipo POSIX, en los que la herencia de variables de entorno a través del árbol de procesos sigue reglas unificadas estándar, Windows combina el uso del depósito seguro de credenciales nativo (Windows Credential Manager), asignaciones en el Registro del sistema y bloques de proceso específicos por consola (PowerShell o depurador de comandos CMD). El desarrollo moderno de agentes — representado por ecosistemas como Google Antigravity (que abarca la aplicación independiente Antigravity 2.0, el entorno IDE, la interfaz Antigravity CLI y el SDK de Python) y Google Agent Development Kit (ADK)— exige un análisis detallado sobre cómo se capturan los tokens de API desde la clave global del sistema y cómo se transmiten hacia subagentes subordinados o servidores de Protocolo de Contexto de Modelo (Model Context Protocol o MCP).

Extracción y Persistencia de Tokens en el Agente Principal

La captura inicial de tokens de API por parte del agente primario en Windows oscila entre el acceso a almacenes criptográficos seguros y la lectura de bloques de entorno de proceso. Cada enfoque introduce requisitos de configuración específicos que determinan si el agente podrá validar sus peticiones ante las API de modelos de lenguaje.

Almacenamiento Seguro en el Keyring Nativo de Windows

El agente primario en arquitecturas como Antigravity CLI (agy) prioritariamente asegura sus tokens de sesión utilizando el depósito seguro de credenciales de la plataforma (system keyring o Windows Credential Manager). Durante el flujo de autenticación mediante cuenta de usuario o Google Cloud, el sistema abre una ventana del navegador predeterminado para resolver el protocolo OAuth o detecta sesiones remotas headless para devolver una URL de autorización. Una vez concedido el acceso, el token opaco resultante se escribe en el almacén seguro de Windows, permitiendo que las ejecuciones subsecuentes del agente reutilicen la sesión sin almacenar secretos en texto plano en el disco duro o en variables de entorno globales.

La Exigencia de Configuración Dual para Claves de API Generales

Cuando se configuran claves de API directas para modelos Gemini mediante la variable de entorno GEMINI_API_KEY, el entorno de Windows requiere una condición adicional para que el agente reconozca el token. A diferencia de utilidades simplificadas que leen automáticamente la variable del sistema, el motor de ejecución exige definir de manera explícita el proveedor de modelos como gemini dentro del archivo de configuración del usuario situado en la ruta del perfil local `~/.gemini/antigravity-cli/settings.json`. Si la variable GEMINI_API_KEY se encuentra presente en la sesión de PowerShell o CMD pero la propiedad `modelProvider` no ha sido establecida explícitamente en el archivo JSON, el agente ignorará la clave de API y el

proceso de autenticación fallará. Inyección de Variables mediante Modificadores del Proceso en Python En el desarrollo de agentes programáticos mediante paquetes como google-adk o marcos de orquestación tipo CrewAI, el agente principal acostumbra a extraer credenciales leyendo archivos de configuración .env a través de librerías como python-dotenv o mediante el gestor de paquetes uv. En estas implementaciones, la función load_dotenv() inyecta los pares clave-valor directamente en el diccionario os.environ del proceso Python en ejecución. Sin embargo, al interactuar con servicios administrados de infraestructura como Agent Engine de Google Cloud, el motor del agente implementa comportamientos de bloqueo. Por ejemplo, Agent Engine reserva la variable de entorno GOOGLE_CLOUD_LOCATION, impidiendo que sea sobrescrita mediante los parámetros habituales de despliegue. Para evitar que la inicialización del motor sobrescriba la ubicación de los modelos Gemini, el código del agente debe interceptar la variable del entorno mediante os.environ.get() antes de instanciar la clase base AdkApp, invocando posteriormente el método superior de configuración y reajustando manualmente el valor capturado.

Herencia de Contexto y Transferencia de Tokens a Subagentes Cuando el agente primario delega subprocesos o subdivide tareas entre múltiples subagentes especializados, la transferencia de tokens de API y variables de entorno se rige por mecanismos de propagación de procesos e inyección explícita de mapas de configuración.

Propagación por Bloque de Proceso y Subprocesos Hijos En el Antigravity SDK para Python, los subagentes se instancian como agentes hijos provistos de conjuntos independientes de herramientas, instrucciones y contextos de sesión. En el sistema operativo Windows, cuando el runtime local engendra un subagente dentro del mismo árbol de ejecución o invoca un subproceso dependiente, el subagente hereda automáticamente el bloque de entorno de proceso (parent process environment block). Esto significa que cualquier token o variable de entorno cargada previamente por el proceso padre estará disponible de forma transparente para los subagentes en ejecución. Adicionalmente, el agente primario puede anular parámetros de conexión a la API instanciando clientes personalizados. En configuraciones avanzadas del SDK, el cliente de API se define utilizando fábricas de tipo Singleton o propiedades protegidas. Al configurar directamente la dirección URL base de la API (base_url) dentro del objeto cliente, el subagente redirige sus solicitudes de inferencia directamente al punto de enlace especificado, omitiendo la consulta de variables de entorno locales, los servidores de metadatos o las restricciones de ámbito regional impuestas por el sistema operativo.

Inyección de Credenciales en Servidores de Protocolo de Contexto de Modelo El Protocolo de Contexto de Modelo (MCP) constituye el estándar mediante el cual los agentes y subagentes se conectan a bases de datos, herramientas del desarrollador o servicios remotos como Google Cloud Run. En Windows, la transmisión de tokens hacia un servidor MCP de comunicación estándar de entrada/salida (stdio) no se realiza por

herencia difusa del entorno global, sino a través de una estructura declarativa explícita. Los servidores MCP se configuran mediante un objeto delimitado en archivos JSON, donde se definen las variables de entorno exclusivas que serán inyectadas al proceso del servidor. Dichas configuraciones se gestionan en dos niveles de alcance: Alcance Global: Ubicado en la ruta de usuario %USERPROFILE%\gemini\config\mcp_config.json, afectando a todas las herramientas de la plataforma. Alcance de Proyecto: Ubicado en la carpeta raíz del espacio de trabajo en .agents\mcp_config.json, circunscribiendo las credenciales únicamente a dicho directorio. Dentro de esta estructura declarativa, el mapa denominado env asigna directamente los tokens requeridos por el proceso secundario, mientras que la propiedad de tipo de proveedor de autenticación (authProviderType) puede ajustarse en google_credentials para facultar al servidor MCP a utilizar automáticamente las Credenciales Predeterminadas de la Aplicación (Application Default Credentials o ADC) del entorno Windows.

Particularidades y Resoluciones Técnicas en el Entorno Windows

La ejecución de arquitecturas de agentes en Microsoft Windows presenta discrepancias estructurales con respecto a plataformas basadas en UNIX, las cuales pueden interrumpir la captura de credenciales o impedir la ejecución de subagentes especializados si no se aplican las correcciones adecuadas.

La Incompatibilidad de la Variable HOME en Subagentes de Navegación Web

Un obstáculo relevante en el entorno Windows se manifiesta al utilizar subagentes con capacidades de navegación web (como la herramienta invocada mediante el comando /browser), los cuales emplean automatización basada en Playwright y motores de renderizado Chromium. Mientras que el sistema operativo Windows utiliza tradicionalmente la variable de entorno %USERPROFILE% para hacer referencia al directorio del usuario, Playwright y diversas herramientas desarrolladas originalmente para ecosistemas POSIX buscan exclusivamente la variable HOME. Al no encontrarse definida de forma predeterminada en Windows, la librería de automatización no logra identificar la ruta para guardar ni ejecutar los archivos binarios de Chromium. Como consecuencia, la creación del contexto del navegador falla, impidiendo que el subagente realice autenticaciones web, capture tokens de sesión o complete flujos de autorización OAuth. Para resolver esta limitación en PowerShell, se requiere registrar permanentemente la variable HOME vinculándola al perfil de usuario activo: `PowerShell[Environment]::SetEnvironmentVariable("HOME", $env:USERPROFILE, "User")`

Tras aplicar esta modificación y reiniciar la consola o el entorno de desarrollo integrado, las herramientas del subagente reconocen la ruta de caché en la carpeta de usuario, permitiendo la descarga de los binarios ejecutables de automatización y garantizando la recolección exitosa de tokens de sesión mediante navegación web.

Normalización de Rutas del Sistema de Archivos y Motor de Permisos

El motor de permisos unificado presente en

los entornos de agentes evalúa cada operación sensible que intenta realizar un agente o subagente a través de reglas estructuradas bajo el formato de acción y objetivo. Las acciones se contrastan frente a tres listas de acceso: denegación (Deny), consulta interactiva (Ask) y aprobación automática (Allow). Debido a que el sistema de archivos de Windows utiliza barras invertidas y letras de unidad (por ejemplo, C:\proyectos\clave.json), el evaluador de seguridad aplica un proceso previo de normalización antes de contrastar la solicitud del agente. El motor remueve automáticamente la letra de la unidad y convierte las barras invertidas en barras diagonales hacia adelante (/). De este modo, la lectura de una clave guardada en C:\proyectos\clave.json se evalúa como /proyectos/clave.json. Esta abstracción asegura que las reglas de seguridad configuradas para la lectura de archivos de credenciales se apliquen de manera uniforme y consistente en Windows.

Sandboxing y Aislamiento de Terminal en Windows Cuando el entorno de ejecución habilita el aislamiento de terminal (Terminal Sandboxing), las autorizaciones concedidas por el usuario alimentan de forma dinámica los recursos del sistema. Si un subagente recibe permiso bajo la regla read_file sobre una ruta que contiene tokens de API, dicho directorio se incorpora a la lista de lectura del sistema de archivos aislado. Asimismo, la concesión de acceso a dominios mediante la regla read_url determina las políticas de red saliente permitidas para el agente. Las acciones no configuradas explícitamente en el espacio de trabajo revierten por defecto al estado de consulta interactiva (Ask), garantizando que el subagente no pueda extraer tokens fuera de la carpeta del proyecto sin solicitar la aprobación explícita del usuario.

Matriz Comparativa de Inyección de Tokens y Comportamiento de Subagentes La siguiente tabla resume los diferentes mecanismos mediante los cuales un agente o subagente captura tokens de API en Windows, detallando el ámbito de almacenamiento, la forma de inyección y el comportamiento de herencia entre procesos.

Componente o Marco	Fuente del Token de API	Mecanismo de Inyección en Windows	Modo de Herencia por Subagentes	Consideraciones Técnicas en Windows
Antigravity CLI (agy)	Depósito seguro de Windows (Keyring) o OAuth	Autenticación interactiva en navegador o ajuste en settings.json [cite: 2, 8]	Herencia de credenciales gestionada por el arnés principal	Requiere definir modelProvider: gemini para procesar la variable GEMINI_API_KEY.
Antigravity SDK (Python)	Variables de entorno del proceso / ADC de Google Cloud	Carga dinámica vía python-dotenv en la sesión de Python	Herencia directa del bloque de entorno del proceso padre	Soporta configuración de niveles de razonamiento (Thinking levels) y eventos de hooks.
Servidor MCP (Stdio)	Mapeo explícito de credenciales en archivo JSON	Declaración del mapa env en mcp_config.json global o local	Inyección aislada al subprocesso del servidor MCP	Permite conectar credenciales de Google ADC fijando authProviderType en google_credentials.Google ADK (google-adk)
Vertex AI / Clientes de Google GenAI	Inyección en el constructor Client o variables del sistema	Propagación a través de clases derivadas		

(AdkApp)Exige la captura y restauración manual de variables como GOOGLE_CLOUD_LOCATION.Subagente de Navegación WebCookies de sesión / Tokens extraídos vía ChromiumDefinición de la variable de usuario HOME en PowerShellCompartición del contexto del navegador y cachés localesSin la variable HOME asignada a %USERPROFILE%, la inicialización de Playwright se bloquea.Conclusiones y Recomendaciones para Entornos de ProducciónLa gestión exitosa de tokens de API por parte de agentes y subagentes en entornos Microsoft Windows requiere una combinación de prácticas de aislamiento criptográfico, configuración de variables de entorno y ajuste de parámetros del sistema operativo.En primer lugar, es crucial evitar la dependencia exclusiva de variables de entorno globales del sistema para almacenar tokens sensibles en texto plano. La utilización del almacén de credenciales nativo de Windows o el uso de configuraciones MCP circunscritas al ámbito del proyecto (.agents\mcp_config.json) proporciona una barrera de contención efectiva que impide la inspección no autorizada por parte de otros procesos en ejecución.En segundo lugar, la preparación del entorno de ejecución en Windows debe formar parte de los scripts de despliegue o bootstrap de la infraestructura. La asignación explícita de la variable de entorno HOME en el perfil del usuario mediante PowerShell garantiza que los subagentes especializados en tareas de navegación web o integración con herramientas POSIX ejecuten sus componentes sin errores de localización de ruta.Finalmente, al orquestar arquitecturas complejas con múltiples subagentes, la definición rigurosa del motor de permisos mediante patrones de coincidencia normalizados y el establecimiento explícito de los proveedores de modelos en los archivos de configuración JSON previenen fallos de autenticación silenciosos y aseguran la trazabilidad completa del consumo de tokens en todo el ciclo de vida del agente.